# FamilyForge — User Manual

A pedigree-aware clinical variant-interpretation tool (annotated VCF + pedigree + panel → ranked candidates + clinician / publication / clinical reports)

Bright Adu

Genomics & Bioinformatics Core
Noguchi Memorial Institute for Medical Research
University of Ghana, Legon, Ghana

2026-07-30

# Contents


**1. Introduction** **3**

**2. Concepts** **4**
2.1 The pedigree / phenotype model . . . 4
2.2 Genotype quality gating . . . 4
2.3 Inheritance models . . . 5
2.4 Monozygotic-twin handling . . . 5
2.5 ACMG-style tiering (A / B / C) . . . 5
2.6 CNV interpretation . . . 6
2.7 Genotype-based relatedness & sex QC . . . 6

**3. Architecture** **7**

**4. Installation** **9**
4.1 pip + virtualenv (recommended for a single user) . . . 9
4.2 conda / mamba (recommended when you also want bcftools) . . . 9
4.3 Verify the installation . . . 9

**5. Inputs** **10**
5.1 The annotated VCF (`--vcf`, required) . . . 10
5.2 The pedigree (`--ped`, required) . . . 10
5.3 The gene panel (`--panel`, required) . . . 10
5.4 Configuration (`--config`, optional) . . . 11
5.5 CNV calls (`--cnv`, optional) . . . 11
5.6 Finalist in-silico scores (`--extra-scores`, optional) . . . 11
5.7 Provenance (`--provenance`, optional) . . . 11

**6. The `familyforge` command** **12**

**7. The `familyforge-clinical-report` command** **14**

**8. Configuration** **16**

**9. Interpretation logic in detail** **18**

**10. Outputs** **20**

**11. Report templates & the clinical letterhead** **21**

**12. Worked example — the self-test** **22**

**13. Troubleshooting** **23**

**14. Reproducibility & provenance** **24**

**15. Limitations** **25**

16. Glossary 26

17. References & citation 27

# 1. Introduction

**FamilyForge** is a pure-Python, disease-agnostic **clinical variant-interpretation tool**. It takes a single family's annotated multi-sample VCF, a pedigree, and a curated gene panel, and produces **segregation-checked candidate variants**, a pragmatic **ACMG-style A/B/C tiering**, a **copy-number (CNV) cross-check**, genotype-based **relatedness / sex QC**, and a set of **audience-specific reports** (a concise clinician report, a full publication report, and a letterhead clinical-genomics report).

FamilyForge is the **interpretation** member of the *forge* family of sibling pipelines:

> **CaptureForge** (probe *design*) → **CallForge** (sequence *analysis*) → **FamilyForge** (family *interpretation*).

CallForge turns demultiplexed FASTQs into a joint-genotyped, annotated callset; FamilyForge consumes that callset — specifically the annotated joint `VCF.gz`, and optionally CallForge's `cnv_calls.tsv` and `provenance.json` — and answers the clinical question CallForge does not: *given this family and this panel, which variants best explain the affected members' disease, and how strong is the evidence?*

**What makes it "family-aware."** FamilyForge never scores a variant in isolation. Every candidate is tested against the **affected / unaffected structure of the pedigree** under explicit Mendelian models, cross-checked for **monozygotic-twin concordance**, and — where a parent was not sequenced — labelled honestly as *inferred, not confirmed*. Population frequency thresholds are applied differently for dominant versus recessive hypotheses, and the tiering rubric folds in ClinVar, predicted consequence, in-silico damage, conservation, and the gene's known disease mechanism.

**Who it is for.** Clinical bioinformaticians, medical geneticists, and genomics-core staff who need a reproducible, honest first pass over a small family's exome/genome data — one that ranks candidates, shows its evidence trail, and produces documents suitable for a multidisciplinary review or a case report.

> **Scope and status.** FamilyForge is a **research** tool. Its outputs are *candidates*, not diagnoses: every Tier A/B finding is explicitly pending orthogonal confirmation (Sanger / MLPA / aCGH), functional validation, and formal ACMG classification by an accredited laboratory before any clinical action. It is **implemented and verified end-to-end on a synthetic self-test** (§12). The bundled gene-specific tiering rules and example configuration were developed around an inborn-error-of-immunity use case, but the tool is **fully disease-agnostic**: the **panel and pedigree are inputs**, never hardcoded, and all report wording (disease name, panel name, per-gene clinical notes) is supplied through configuration — nothing disease-specific is baked into the templates (§11).

# 2. Concepts

This section defines the ideas the tool is built on. It is worth reading once before the command reference, because the flags map directly onto these concepts.

## 2.1 The pedigree / phenotype model

FamilyForge represents a family as a set of **members**, each parsed from one line of a standard **6-column PED** file:

| Column | Meaning | Notes |
|---|---|---|
| 1 | Family ID | free text (e.g. `FAM01`) |
| 2 | Individual ID (`iid`) | **must match the VCF sample name** for a sequenced member |
| 3 | Father ID | `0` if unknown/founder |
| 4 | Mother ID | `0` if unknown/founder |
| 5 | Sex | `1`/`M`=male, `2`/`F`=female, `0`/`U`=unknown |
| 6 | Phenotype | `2` = affected; anything else = unaffected |

A crucial design choice: a member is marked **sequenced** *if and only if* its `iid` appears among the VCF sample names. Members present only to define relationships — most commonly an **unsequenced father** — are kept in the pedigree so the models can reason about obligate carriers and phase, but they contribute no genotypes. This is what lets FamilyForge state, for a de-novo-pattern variant, "*de novo OR paternally inherited (unconfirmed)*" rather than over-claiming.

Derived quantities the models use: `affected` (sequenced affected members), `unaffected` (sequenced unaffected members), and `mothers_of_affected()` (the set of mothers, present in the pedigree, of the affected members — the obligate-carrier check for recessive models).

## 2.2 Genotype quality gating

Before any inheritance model runs, each per-sample genotype is gated. A het, hom-alt, or hemizygous-alt call is demoted to "not usable" (`ok = False`, treated like a no-call by the models) when it fails the configured thresholds:

- genotype quality `GQ < min_gq` (default **20**), or
- depth `DP < min_dp` (default **8**), or
- for a heterozygous call, alt allele-balance `AB < min_alt_ab` (default **0.20**).

Homozygous-alt allele balance is handled leniently (a small residual REF fraction does not by itself void a hom-alt call). Zygosity itself is computed allele-count-wise, with **hemizygous** recognised on chrX/chrY for male samples. This gate is the reason a noisy low-depth genotype cannot, on its own, manufacture a de-novo or compound-het call.

## 2.3 Inheritance models

FamilyForge evaluates four single-variant / gene-level Mendelian models. Each is checked against the affected/unaffected structure; a model only fires if the panel's declared inheritance for the gene is compatible (or the panel leaves inheritance blank, in which case any model may fire).

- **De novo / dominant** (`de_novo_or_dominant`). All sequenced affected carry the allele (het), all sequenced unaffected are homozygous-reference, and the gene allows `AD` (or has no declared inheritance). If an unaffected relative also carries the allele, a separate *reduced-penetrance* dominant branch fires instead, at low confidence and flagged.
- **Autosomal-recessive homozygous** (`ar_homozygous`). All affected are homozygous-alt on an autosome, no unaffected sibling is homozygous, and (where sequenced) the mother is not homozygous-alt — an obligate-carrier mother is expected to be heterozygous. Confidence is *high* only when the twin genotypes are concordant and every sequenced mother is het.
- **Autosomal-recessive compound heterozygous** (`ar_compound_het`). Evaluated per gene: two different heterozygous variants, both het in *all* affected, on opposite parental haplotypes (*in trans*). Phase is inferred from the mother — one allele maternal (mother het), the other absent in the mother (mother hom-ref, therefore inferred paternal). No unaffected relative may carry **both** alleles. When the father is unsequenced, the paternal allele is labelled *inferred*.
- **X-linked recessive** (`x_linked_recessive`). On chrX, affected males are hemizygous for the allele, no sister is homozygous, the mother (where sequenced) is a heterozygous carrier, and the gene allows `XLR`/`XL`.

Each model emits a human-readable **segregation string** — e.g. `MOTHER(unaff)=het; SIBLING(unaff)=ref; TWIN_1(aff)=hom; TWIN_2(aff)=hom` — so a reviewer can see the family pattern at a glance.

## 2.4 Monozygotic-twin handling

If two or more samples are declared an **MZ-twin group** (via `--mz-twins` or config), they are treated as *genetic replicates*: for a candidate to reach high confidence, the twins must be **genotype-concordant** at that site. A discordant genotype across an MZ pair is almost always a technical artifact (a dropout or a miscall), so the candidate is flagged and its confidence capped — and in tiering, a low-confidence Tier A is demoted to Tier B. Independently, the genotype-QC step reports pairwise concordance across the whole panel, so you can *verify* that a declared MZ pair really behaves like one ($\approx$1.0) and that first-degree relatives sit near 0.5–0.7.

## 2.5 ACMG-style tiering (A / B / C)

FamilyForge assigns each candidate a **tier**, a pragmatic triage signal — *not* a full, formal ACMG classification:

- **Tier A — likely actionable.** ClinVar pathogenic/likely-pathogenic (rare, segregating); or biallelic predicted-LoF in a LoF-mechanism gene under a recessive model; or a STAT3 dominant-negative hotspot missense/LoF under a de-novo/dominant pattern; or a rare damaging missense in a Tier-1/2 gene with strong in-silico consensus (AlphaMissense likely-pathogenic, REVEL $\geq 0.7$, or CADD $\geq 25$) under a matching-inheritance model.

- **Tier B — strong VUS.** Predicted-LoF in a panel gene, or a rare missense with supportive in-silico evidence, where mechanism or inheritance still needs confirmation. Tier A findings are **capped to B** when ClinVar is *conflicting*, or when genotype confidence is low / MZ-twin discordant.
- **Tier C — weaker / downgraded.** Rare in-panel variants with limited support, or variants downgraded because they are too common in gnomAD for their model, or ClinVar benign.

The evaluation is top-to-bottom, first-match-wins; §9 and the tiering diagram (Figure 2) document every branch. Alongside the tier, FamilyForge records an **evidence trail** — the list of criteria that fired (ClinVar status, consequence, AF, SIFT/PolyPhen/PhyloP, AlphaMissense/REVEL/CADD, segregation model) — which is printed verbatim in the reports.

A separate `classify()` step maps each row to a five-level **colour convention** for the reports — **P** (pathogenic), **LP** (likely pathogenic), **VUS**, **LB** (likely benign), **B** (benign) — from ClinVar, tier, and allele frequency.

## 2.6 CNV interpretation

SNV/indel calling is blind to large deletions, yet deletions are a classic disease mechanism for some genes (the bundled example is **DOCK8** in AR-HIES). FamilyForge reads CallForge's `cnv_calls.tsv` (tolerant of column-name variations), keeps **deletions that overlap a panel gene**, and tiers them: **Tier A** if the gene is a configured CNV-priority gene *and* the event is in an affected sample, otherwise **Tier B**. Low-confidence CNV labels and the SNV-blind mechanism are surfaced as flags, and every CNV candidate carries the standing recommendation to confirm by read-depth/IGV and an orthogonal assay (MLPA/aCGH).

## 2.7 Genotype-based relatedness & sex QC

Independently of any external sites file (no somalier required), FamilyForge cross-checks the samples from their own genotypes:

- **Pairwise concordance** over shared, quality autosomal sites — a relatedness / MZ / sample-swap check.
- **Sex inference** from chrX heterozygosity — a sample with a very low chrX het-rate ($< 0.10$) is inferred male, otherwise female — compared against the declared sex in the PED.

These land in every report's QC section and in `candidates.json`.

# 3. Architecture

FamilyForge is a small, readable Python package. A single command (`familyforge`) orchestrates the whole run; a second entry point (`familyforge-clinical-report`) renders the letterhead clinical report from the machine-readable output. The figure below is the authoritative map: inputs (top) flow through ingest/parse, into the interpretation modules, into reporting, and out to the deliverables.

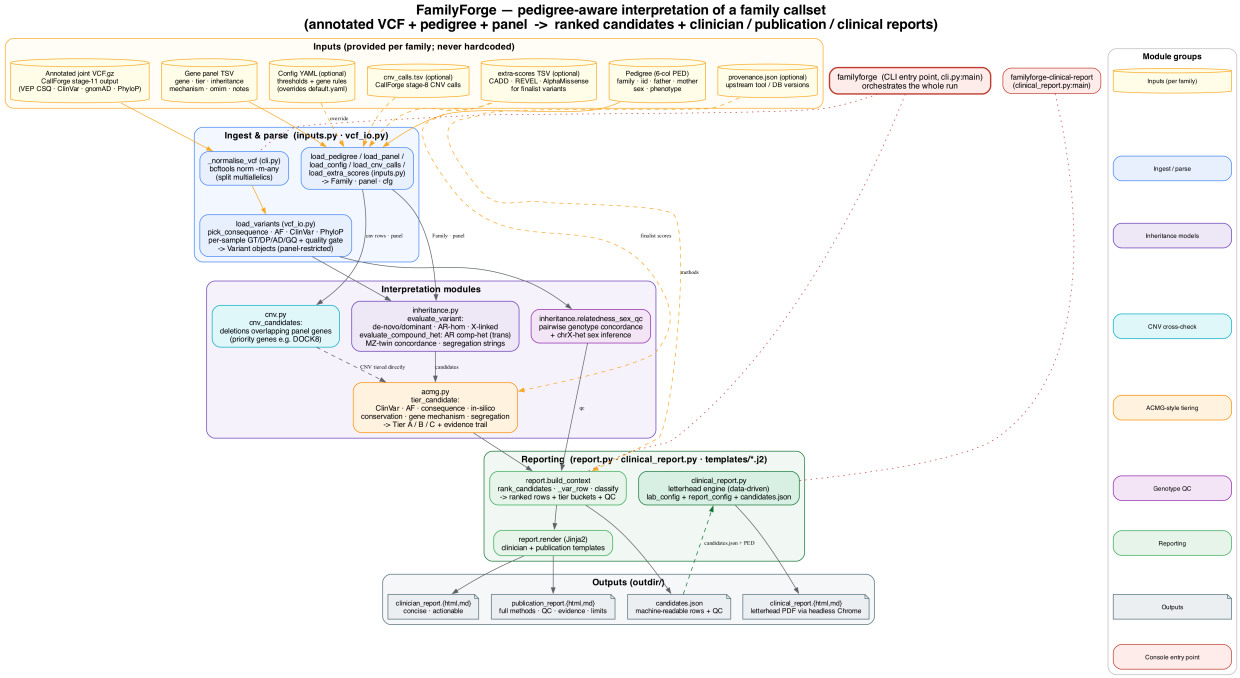



**Figure 1:** FamilyForge architecture — inputs (annotated VCF, pedigree, panel, optional config / CNV / extra-scores / provenance) are parsed by `vcf_io.py` and `inputs.py`, interpreted by `inheritance.py`, `cnv.py`, and `acmg.py`, then rendered by `report.py` and the `clinical_report.py` letterhead engine into clinician, publication, clinical, and JSON outputs.

**Module map.**

| File | Responsibility |
|---|---|
| `cli.py` | Argument parsing, orchestration, VCF normalisation, JSON dump |
| `inputs.py` | Loaders: config (YAML), pedigree (PED), panel (TSV), CNV calls, extra scores |
| `vcf_io.py` | Parse the annotated VCF into `Variant` objects; pick consequence; quality-gate genotypes |
| `model.py` | Dataclasses: `Member`, `Family`, `GeneAnno`, `Variant`, `Candidate` |
| `inheritance.py` | Mendelian models, compound-het, MZ concordance, relatedness/sex QC |

| File | Responsibility |
|---|---|
| `acmg.py` | ACMG-style A/B/C tiering with an evidence trail |
| `cnv.py` | CNV deletion cross-check against panel genes |
| `report.py` | Build the report context; render clinician + publication (HTML + MD) |
| `clinical_report.py` | Standalone letterhead clinical-report engine (HTML + MD) |
| `config/default.yaml` | Default thresholds and gene-specific rules |
| `templates/*.j2` | Jinja2 templates for all four report styles |
| `tests/selftest.py` | Synthetic end-to-end test with planted, known-answer variants |

**Data-flow spine** (as in `cli.main`): normalise VCF → load panel → discover which pedigree members are in the VCF → `load_variants` (panel-restricted, quality-gated) → apply optional `--extra-scores` → run `evaluate_variant` + `evaluate_compound_het` + `cnv_candidates` → `tier_candidate` each SNV/indel candidate → `relatedness_sex_qc` → `build_context` → `render` (four report files) → write `candidates.json`.

# 4. Installation

FamilyForge is a standard Python package (build backend: setuptools). It needs **Python ≥ 3.10** and five pure-Python/compiled dependencies: `cyvcf2`, `pandas`, `numpy`, `jinja2`, `pyyaml`. `cyvcf2` provides fast VCF parsing (it ships binary wheels for common platforms). **`bcftools` on `PATH`** is recommended but optional — it is used to split multiallelic sites; without it, multiallelic records are skipped with a warning.

## 4.1 pip + virtualenv (recommended for a single user)

```
# 1. obtain FamilyForge and enter the repository
git clone https://github.com/aduton1000/familyforge.git
cd familyforge

# 2. create and activate an isolated environment
python3 -m venv .venv
source .venv/bin/activate        # Windows: .venv\Scripts\activate

# 3. install (editable so local edits take effect immediately)
pip install -e .

# 4. confirm the console entry points are on PATH
familyforge --help
familyforge-clinical-report --help
```

## 4.2 conda / mamba (recommended when you also want bcftools)

```
mamba create -n familyforge -c conda-forge -c bioconda \
    python=3.11 cyvcf2 pandas numpy jinja2 pyyaml bcftools
mamba activate familyforge
pip install -e .          # register the familyforge console commands
```

## 4.3 Verify the installation

Run the bundled self-test (§12). It fabricates an annotated VCF with planted variants and confirms FamilyForge recovers the expected models, tiers, and reports:

```
python tests/selftest.py
# expect: Tier A genes: {'STAT3', 'DOCK8', 'PGM3'} ... SELFTEST PASS
```

# 5. Inputs

FamilyForge takes one required trio — **VCF + PED + panel** — plus four optional inputs. All are provided at run time; nothing is hardcoded.

## 5.1 The annotated VCF (`--vcf`, required)

A multi-sample, bgzipped VCF — normally CallForge's stage-11 `annotated.vcf.gz`. FamilyForge reads:

- **Per-sample FORMAT**: `GT`, `DP`, `AD`, `GQ` (for zygosity + quality gating).
- **VEP `CSQ`** (INFO): `Consequence`, `SYMBOL`, `HGVSc`, `HGVSp`, `SIFT`, `PolyPhen`, `CANONICAL`, `MANE_SELECT`/`MANE`, and the population-AF fields listed in `af_fields`.
- **INFO annotations**: `ClinVar_CLNSIG` (or `CLNSIG`, or the CSQ `CLIN_SIG` field), `PhyloP`, and `PARALOG_GENE`/`PARALOG_CONF` if present.

For each variant, FamilyForge **picks one transcript consequence** — preferring a panel gene, then a MANE/canonical transcript, then the most-severe consequence — and, in its primary pass, **keeps only variants whose picked gene is in the panel**. The MANE Select RefSeq accession is retained so the reports can present clinical-standard HGVS.

Multiallelic handling: FamilyForge runs `bcftools norm -m-any` first (unless `--skip-norm`), so genotype interpretation is unambiguous. If a record is still multiallelic at parse time, it is skipped.

## 5.2 The pedigree (`--ped`, required)

A 6-column PED (§2.1). Sample IDs in column 2 must match the VCF sample names for sequenced members. Example (`family.ped`) — an unsequenced father, an unaffected mother, an unaffected sibling, and affected MZ twins:

```
FAM01  FATHER   0       0       1  0
FAM01  MOTHER   0       0       2  1
FAM01  SIBLING  FATHER  MOTHER  2  1
FAM01  TWIN_1   FATHER  MOTHER  1  2
FAM01  TWIN_2   FATHER  MOTHER  1  2
```

Here `FATHER` has no genotypes in the VCF, so FamilyForge marks him *relationship-only*; `TWIN_1`/`TWIN_2` are the affected (phenotype `2`) males.

## 5.3 The gene panel (`--panel`, required)

A tab-separated file, one gene per row, with a header line beginning `gene`. Columns after the gene are optional; missing columns default sensibly.

```
gene    tier    inheritance    mechanism    omim      notes
STAT3   1       AD             DN           147060    DBD/SH2 hotspot incl R382
DOCK8   1       AR             LOF          243700    biallelic LOF, large
 ↪ deletions
```

```
PGM3    2       AR              HYPO          615816    hypomorphic/LOF
IL2RG   1       XLR             LOF           300400    X-linked SCID/IEI
```

- **tier** — the gene's a-priori priority (`1/2` = Tier-1/2 disease gene; used by the strong-missense Tier A branch). Defaults to `3`.
- **inheritance** — a comma-separated set of `AD`, `AR`, `XLR`/`XL`; gates which models may fire. Leave blank to allow any model.
- **mechanism** — free set (e.g. `LOF`, `DN`, `HYPO`); informational, surfaced in reports.
- **omim**, **notes** — free text for the reports.

## 5.4 Configuration (`--config`, optional)

A YAML file that **overrides** keys in the packaged `config/default.yaml` (§8). Supply only the keys you want to change.

## 5.5 CNV calls (`--cnv`, optional)

A CallForge `cnv_calls.tsv`. The parser is tolerant of column-name variants: it recognises `gene/genes/symbol/overlapping_genes`, `call/type/cnv/event/status`, `cn/copy_number/log2`, `sample/sample_id`, `chrom/chr`, `start`, `end`, `callability/confidence/label`. A row counts as a deletion if its call text says del/loss/deletion, or copy number `< 1.5`, or log2-ratio `< -0.4`.

## 5.6 Finalist in-silico scores (`--extra-scores`, optional)

When genome-wide dbNSFP is unavailable, you can attach externally retrieved scores for a handful of finalist variants. A TSV keyed by `chrom:pos:ref>alt`:

```
key                     cadd_phred    revel    alphamissense_score
 ↪  alphamissense_class
chr17:42322000:G>A      29.4          0.86     0.978
 ↪  likely_pathogenic
chr9:300000:C>T         34.0          .        .                       .
```

Matching rows have their CADD/REVEL/AlphaMissense fields populated before tiering, which can lift a strong missense to Tier A.

## 5.7 Provenance (`--provenance`, optional)

A CallForge `provenance.json`. Its contents (tool/DB versions, git commit) are threaded into the publication report's methods section for reproducibility.

# 6. The `familyforge` command

`familyforge` is the primary entry point. It runs the full interpretation in one call.

```
familyforge --vcf VCF --ped PED --panel PANEL --outdir DIR
            [--cnv TSV] [--config YAML] [--study-name NAME]
            [--mz-twins "ID1,ID2" ...] [--provenance JSON]
            [--extra-scores TSV] [--skip-norm]
```

**Flag reference.**

| Flag | Req. | Purpose |
|---|---|---|
| `--vcf` | Yes | Annotated joint VCF.gz (CallForge stage-11 output) |
| `--ped` | Yes | 6-column PED pedigree |
| `--panel` | Yes | Gene-panel TSV (`gene,tier,inheritance,mechanism,omim,n` |
| `--outdir` | Yes | Output directory (created if absent) |
| `--cnv` | | CallForge `cnv_calls.tsv` for the CNV cross-check |
| `--config` | | Override YAML (merged over `config/default.yaml`) |
| `--study-name` | | Report title (default `Family study`) |
| `--mz-twins` | | Comma-separated MZ-twin sample IDs; **repeatable** for multiple twin sets |
| `--provenance` | | CallForge `provenance.json` for the methods section |
| `--extra-scores` | | TSV of finalist CADD/REVEL/AlphaMissense scores |
| `--skip-norm` | | Skip `bcftools norm` (use if the VCF is already biallelic-split) |

**Generic example.**

```
familyforge \
  --vcf        /path/to/callforge/anno/annotated.vcf.gz \
  --ped        /path/to/config/family.ped \
  --panel      /path/to/config/gene_panel.tsv \
  --cnv        /path/to/callforge/cnv/cnv_calls.tsv \
  --config     /path/to/config/familyforge.yaml \
  --extra-scores /path/to/config/finalist_scores.tsv \
```

```
  --provenance /path/to/callforge/provenance.json \
  --mz-twins   TWIN_1,TWIN_2 \
  --study-name "Family FAM01 exome" \
  --outdir     /path/to/results/reports
```

**What it prints** (to stderr): the number of panel variants loaded and samples seen, how many extra scores matched, and the tier counts (`nA / nB / nC`), followed by the list of files written.

# 7. The `familyforge-clinical-report` command

The second entry point renders a **letterhead clinical-genomics report** — the disease- agnostic, data-driven document intended for a clinical file. It consumes the machine- readable `candidates.json` produced by a `familyforge` run, plus the pedigree and two small YAML files, and emits an HTML report (render to PDF with headless Chrome) and a Markdown twin.

```
familyforge-clinical-report --lab-config LAB.yaml --report-config REPORT.yaml
                            --candidates candidates.json --ped PED --out OUT.html
```

| Flag | Purpose |
|---|---|
| `--lab-config` | Reusable lab branding YAML (institute, logos, head-of-lab, confidentiality) |
| `--report-config` | Per-report YAML (accession, specimen, disease name, sample labels, QC rows, management) |
| `--candidates` | The `candidates.json` from a `familyforge` run |
| `--ped` | The same PED (for sex/relationship fallback) |
| `--out` | Output HTML path (a `.md` twin is written alongside) |

The `--lab-config` supplies branding fields (`lab_abbrev`, `institute`, `university`, `lab_name`, `logo_banner`, `logo_lab`, `head_name`, `head_title`, `confidentiality`); the logos are read and embedded as base64 so the HTML is self-contained. The `--report-config` supplies the per-case content: `accession`, `report_type`, `status`, `test`, `specimen`, `reference_genome`, `referral_reason`, `disease_name`, `panel_name`, a `samples` map (labels/relationships/status), optional `qc_rows`, `gene_management`, and `notes`. All clinical content is data — nothing about the disease is baked into the engine.

**Generic example** (render to PDF at the end):

```
familyforge-clinical-report \
  --lab-config    /path/to/config/lab_branding.yaml \
  --report-config /path/to/config/report_meta.yaml \
  --candidates    /path/to/results/reports/candidates.json \
  --ped           /path/to/config/family.ped \
  --out           /path/to/results/reports/clinical_report.html

# render the letterhead HTML to PDF (self-contained; repeating header per page)
chrome --headless=new --no-pdf-header-footer \
       --print-to-pdf=/path/to/results/reports/clinical_report.pdf \
       file:///path/to/results/reports/clinical_report.html
```

The letterhead template repeats the institutional banner at the top of every printed page (via a `<thead>` that Chrome repaints), applies the P→LP→VUS→LB→B colour scale to the primary find-

ing, and appends a technical appendix (QC metrics, methods summary, limitations and disclaimer) with a signature/stamp block for the authorising laboratory head.

# 8. Configuration

All thresholds and gene rules live in `familyforge/config/default.yaml`. Override any subset with `--config your.yaml`; your keys are merged on top of the defaults.

| Key | Default | Meaning |
|---|---|---|
| `min_gq` | 20 | Minimum genotype quality; below → genotype not usable |
| `min_dp` | 8 | Minimum depth at the genotype |
| `min_alt_ab` | 0.20 | Minimum ALT allele-balance to accept a het/hom-alt |
| `max_ref_ab_for_hom` | 0.10 | Max REF fraction tolerated for a hom-alt |
| `af_dominant_max` | 0.0001 | Max popmax AF for a plausible **dominant** causal allele |
| `af_recessive_max` | 0.01 | Max popmax AF for a plausible **recessive** causal allele |
| `af_fields` | (list) | CSQ/INFO AF fields to take the max over (incl. African-ancestry) |
| `lof_consequences` | (list) | Consequences counted as loss-of-function |
| `missense_consequences` | (list) | Consequences counted as missense/protein-altering |
| `splice_region` | (list) | Splice-region consequences |
| `phylop_conserved` | 2.0 | PhyloP above this = conserved position |
| `revel_supportive` | 0.5 | REVEL $\geq$ this = supportive |
| `revel_strong` | 0.7 | REVEL $\geq$ this = strong (PP3-strong-ish) |
| `cadd_high` | 25.0 | CADD PHRED $\geq$ this = high deleteriousness |
| `mz_twin_groups` | `[]` | MZ-twin sample-ID groups (also settable via `--mz-twins`) |
| `mz_min_concordance` | 0.97 | Expected genome-wide MZ concordance |
| `stat3_dbd_range` | `[321,465]` | STAT3 DNA-binding-domain residue range |
| `stat3_sh2_range` | `[585,688]` | STAT3 SH2-domain residue range |
| `stat3_hotspot_residues` | (list) | Individual STAT3 hotspot residues |

| Key | Default | Meaning |
|---|---|---|
| `lof_mechanism_genes` | (list) | Genes whose LoF is the disease mechanism |
| `cnv_priority_genes` | `[DOCK8]` | Panel genes where deletions are especially important |
| `disease_name` | `the suspected condition` | Report wording — the disease under investigation |
| `panel_name` | `custom` | Report wording — the panel's name |
| `gene_clinical_notes` | (map) | Gene → short clinical/counseling note emitted for the top candidate |

The `af_fields` default deliberately includes African-ancestry fields (`gnomADe_AFR_AF`, `gnomADg_AFR_AF`, `gnomAD_AF_afr`) alongside the global ones, so frequency filtering is appropriate for African-ancestry families. The `stat3_*`, `lof_mechanism_genes`, and `cnv_priority_genes` keys are the **gene-specific rules**: they bias the tiering toward known disease mechanisms, and you extend them (or replace them for a different disease area) purely in configuration. The `disease_name`, `panel_name`, and `gene_clinical_notes` keys are the **report-content** settings that make the reports disease-agnostic (§11): they supply the disease/panel names and the per-gene counseling notes shown in the clinician and publication reports, with nothing disease-specific baked into the templates. The shipped `gene_clinical_notes` entries (DOCK8, STAT3) are illustrative examples of the format only.

# 9. Interpretation logic in detail

This section documents exactly how a candidate becomes a tier and an evidence trail. It mirrors `acmg.tier_candidate` and the inheritance models; Figure 2 is the decision map.

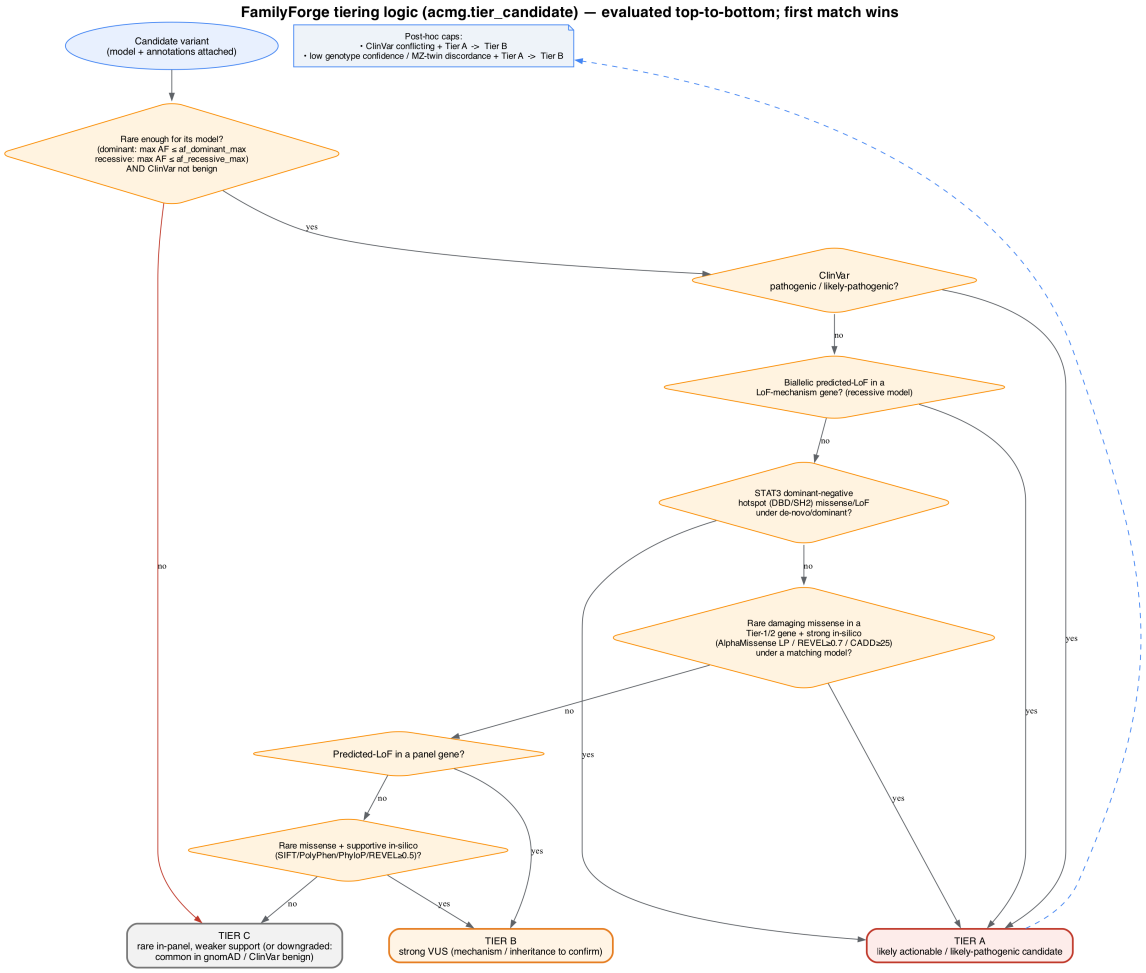



**Figure 2:** FamilyForge tiering decision flow (`acmg.tier_candidate`). Branches are evaluated top-to-bottom and the first match wins; two post-hoc caps demote a Tier A to Tier B under ClinVar-conflicting or low-confidence/MZ-discordant conditions.

**Step 1 — establish the model.** Every candidate arrives already carrying a model (`de_novo_or_dominant`, `ar_homozygous`, `ar_compound_het`, `x_linked_recessive`, or `cnv_deletion`) set by the inheritance/CNV step, together with a segregation string and a confidence (`high`/`moderate`/`low`).

**Step 2 — gather evidence.** For SNV/indel candidates, `tier_candidate` derives: ClinVar class (pathogenic / benign / conflicting / VUS / none), whether the consequence is LoF / missense / splice-region, whether the allele is **rare enough for its model** (using `af_dominant_max` for dominant models and `af_recessive_max` for recessive ones), whether the gene is a LoF-mechanism gene, whether it is a STAT3 hotspot, whether the position is conserved (PhyloP), and whether in-silico support is *supportive* or *strong* (SIFT deleterious, PolyPhen damaging, PhyloP conserved,

AlphaMissense likely-pathogenic, REVEL $\geq$ 0.5/0.7, CADD $\geq$ 25). It records a human-readable **evidence list** used verbatim in the reports.

**Step 3 — decide the tier** (first match wins):

1. **Not rare, or ClinVar benign → Tier C** (downgraded).
2. **ClinVar pathogenic/likely-pathogenic → Tier A** ("rare, segregates").
3. **Biallelic predicted-LoF in a LoF-mechanism gene, recessive model → Tier A.**
4. **STAT3 dominant-negative hotspot missense/LoF, de-novo/dominant → Tier A.**
5. **Rare damaging missense in a Tier-1/2 gene with strong in-silico consensus, under a matching-inheritance model → Tier A** (likely-pathogenic candidate; pending functional + parental confirmation).
6. **Predicted-LoF in a panel gene → Tier B** (mechanism/inheritance to confirm).
7. **Rare missense with supportive in-silico → Tier B.**
8. **Otherwise → Tier C.**

**Step 4 — apply caps.** A Tier A with *conflicting* ClinVar is demoted to Tier B; a Tier A with low genotype confidence (e.g. an MZ-twin discordance) is demoted to Tier B with the reason annotated.

**How the reports read this.** `report.classify()` translates (tier, ClinVar, AF) into the five-level P/LP/VUS/LB/B colour code; `rank_candidates` orders findings by tier, then confidence, then the gene's panel tier, so the single most-defensible finding is always `top`. The clinician and publication templates print the model, zygosity, segregation string, evidence trail, and tier reason; the letterhead engine renders the `top` finding as the primary result with recommendations.

# 10. Outputs

A `familyforge` run writes, into `--outdir`:

| File | Audience | Contents |
|---|---|---|
| `clinician_report.html` / `.md` | Clinician / MDT | Concise: summary, family table, priority (Tier A/B) findings with evidence, clinical implications, QC, caveats |
| `publication_report.html` / `.md` | Manuscript / case report | Full: methods, tiered candidate tables (A/B/C), per-candidate evidence, provisional ACMG-code table, QC tables, panel table, limitations |
| `candidates.json` | Downstream / letterhead engine | Machine-readable: context summary, ranked rows, and QC — the input to `familyforge-clinical-report` |

Running `familyforge-clinical-report` then adds `clinical_report.html` (+ `.md`), which you render to `clinical_report.pdf`.

**Reading a candidate row.** Each row carries: gene, model, tier, confidence, zygosity, segregation string, inheritance note, tier reason, the flags/evidence list, and (for SNV/indels) locus, rsID, filter, consequence, clinical-HGVS (MANE-prefixed), ClinVar, gnomAD (global + AFR), PhyloP, SIFT, PolyPhen, and CADD/REVEL/AlphaMissense where supplied. Compound-het rows also carry the partner allele; CNV rows carry the deletion locus and copy-number.

# 11. Report templates & the clinical letterhead

FamilyForge ships **three report styles**, all as Jinja2 templates under `familyforge/templates/`:

- `clinician.{html,md}.j2` and `publication.{html,md}.j2` — rendered by `report.py`. These are the concise and full research reports.
- `clinical_letterhead.{html,md}.j2` — rendered by `clinical_report.py`, the institutional letterhead document.

**All three report styles are disease-agnostic and config-driven.** No disease, panel, or gene name is hardcoded in any template. The wording is supplied through configuration and the report YAMLs:

- **`clinician / publication` templates** read `disease_name`, `panel_name`, and the `gene_clinical_notes` map from the run config (`config/default.yaml`, overridable with `--config`), threaded into the report context as `meta.disease_name`, `meta.panel_name`, and `meta.gene_clinical_notes`. Titles, the summary line, the panel-name heading, and the phenotype-correlation note all interpolate these values. Per-gene clinical/counseling text is **data-driven**: for the top candidate gene, the template emits the matching `gene_clinical_notes` entry if one exists, and nothing otherwise — so counseling notes are configured, never coded. Missing values degrade gracefully to neutral defaults (`the suspected condition`, `custom`).
- **`clinical_letterhead` template** (letterhead engine) takes its disease name, panel name, sample labels, methods wording, and management text from the `report_config` YAML, exactly as before.

To adapt the reports to your disease area, set `disease_name`, `panel_name`, and `gene_clinical_notes` in a `--config` YAML (and the corresponding fields in the letterhead `report_config`). The bundled defaults ship two illustrative `gene_clinical_notes` entries (DOCK8, STAT3) purely as **examples** of the format; replace them with notes appropriate to your panel. The methods/provenance paragraph retains generic tool-provenance wording (CallForge/FamilyForge/VEP/GATK) with no disease- or gene-specific phrasing.

The letterhead template is designed for print: A4 geometry, a repeating banner header, the P→LP→VUS→LB→B colour scale, a variant-detail table, interpretation and recommendations, a signature/stamp block, and a second-page technical appendix (QC metrics, methods, limitations & disclaimer). Because logos are embedded as base64, the HTML is a single self-contained file that headless Chrome prints faithfully to PDF.

# 12. Worked example — the self-test

The bundled `tests/selftest.py` is the fastest way to see FamilyForge end-to-end. It writes a synthetic annotated VCF for a four-member family (`MOTHER`, `SIBLING`, and the affected MZ twins `TWIN_1`/`TWIN_2`), with an unsequenced `FATHER` in the PED, and plants four known-answer variants plus a benign decoy:

1. **STAT3** de-novo hotspot missense (p.Arg382Trp) — het in both affected, ref in mother/sibling → expected **Tier A**, de-novo/dominant.
2. **DOCK8** homozygous stop-gained — hom in both affected, het in mother/sibling → expected **Tier A**, AR-homozygous.
3. **PGM3** compound-het (a maternal stop-gained + an inferred-paternal frameshift) → expected **Tier A**, AR-compound-het.
4. A common, benign **STAT3** missense → expected to be filtered out (downgraded).

```
python tests/selftest.py
```

Expected result:

```
=== SELFTEST RESULT ===
Tier A genes: {'STAT3', 'DOCK8', 'PGM3'}
summary: {'n_candidates': ..., 'tierA': 3, 'tierB': ..., 'tierC': ...}
...
SELFTEST PASS
```

The test also confirms the four report files are written and non-empty, and that the MZ pair `TWIN_1~TWIN_2` shows high genotype concordance. Inspect the generated `out/` directory to see real clinician, publication, and JSON outputs.

# 13. Troubleshooting

| Symptom | Cause / fix |
|---|---|
| `WARN: bcftools not on PATH - skipping normalisation` | Install `bcftools`, or pass `--skip-norm` if the VCF is already biallelic-split. Without it, multiallelic sites are ignored. |
| `loaded 0 panel variants` | The picked-consequence gene never matched the panel. Check that `SYMBOL` in the VEP `CSQ` uses the same gene symbols as the panel, and that the VCF is actually annotated. |
| A sequenced member shows as *not sequenced* | Its PED `iid` (column 2) does not exactly match a VCF sample name. Names are case-sensitive. |
| No Tier A/B despite an obvious candidate | Check genotype quality (a low-`GQ`/`DP`/`AB` call is gated out), the panel's `inheritance` (an incompatible model will not fire), and the AF thresholds (too-common alleles are downgraded). |
| Expected de-novo call is "unconfirmed" | This is by design when the father is not sequenced — add paternal DNA to confirm. |
| MZ pair flagged discordant / Tier A capped to B | A genotype differs across the twins at that site — usually a technical dropout; review depth/AB, and confirm the twins really are the declared pair via the QC concordance table. |
| CNV rows ignored | The row was not recognised as a deletion (call text / copy-number / log2 thresholds), or its gene did not overlap the panel. Check column names against §5.5. |
| Letterhead engine errors on a missing logo | `logo_banner`/`logo_lab` paths in the lab-config must point to readable image files (they are base64-embedded). |

# 14. Reproducibility & provenance

Reproducibility rests on three things: the **inputs** (VCF, PED, panel, config), the **configuration** (all thresholds and rules are in `config/default.yaml` and echoed into the publication report's methods), and the **upstream provenance** (`--provenance` threads CallForge's tool/DB versions and git commit into the report). The `candidates.json` is a complete, machine-readable record of every ranked candidate and the QC, so a run can be re-examined without re-running the analysis. Pin the package version (`familyforge.__version__`, printed in every report) and the config file alongside your results.

# 15. Limitations

- **Candidates, not diagnoses.** Every Tier A/B finding is pending orthogonal confirmation (Sanger / MLPA / aCGH), functional validation, and formal ACMG classification by an accredited laboratory.
- **Missing parents.** With an unsequenced parent, de-novo status and compound-het phase are *inferred*, not confirmed.
- **Annotation-bound.** Interpretation quality is bounded by the upstream annotation: population-AF completeness (ancestry-specific fields especially), and in-silico coverage. When genome-wide dbNSFP is unavailable, supply finalist scores via `--extra-scores`.
- **Panel-restricted.** The primary pass keeps only variants in panel genes; variants outside the panel are not evaluated.
- **Pragmatic tiering.** The A/B/C rubric is a triage signal, not the full ACMG framework; it does not enumerate every ACMG criterion or compute a formal combined classification.
- **Report content is configured, not inferred.** All report styles are disease-agnostic (§11); the disease/panel names and per-gene counseling notes are supplied via config, so set `disease_name`, `panel_name`, and `gene_clinical_notes` for your disease area rather than relying on the neutral defaults.
- **CNV depth.** CNV cross-check surfaces deletions overlapping panel genes from an external caller's output; it does not itself call CNVs and requires orthogonal confirmation.

# 16. Glossary

- **ACMG** — American College of Medical Genetics variant-classification framework; FamilyForge implements a pragmatic *subset* for triage.
- **AF / popmax** — population allele frequency; the maximum across the configured `af_fields`.
- **Compound heterozygous (in trans)** — two different heterozygous variants in the same gene on opposite parental haplotypes.
- **De novo** — a variant present in the child but neither parent; here often *inferred* when a parent is unsequenced.
- **Hemizygous** — a single-copy genotype (males on chrX/chrY).
- **HGVS** — standard nomenclature for cDNA (`c.`) and protein (`p.`) changes; reports prefer the MANE Select RefSeq transcript.
- **LoF** — loss-of-function (stop-gained, frameshift, canonical splice, etc.).
- **MANE Select** — the jointly agreed representative transcript per gene (NCBI/EMBL-EBI).
- **MZ twins** — monozygotic (identical) twins; treated as genetic replicates.
- **Segregation** — how a variant's genotypes track the affected/unaffected pattern in the family.
- **VUS** — variant of uncertain significance.

# 17. References & citation

**Cite FamilyForge** using the repository `CITATION.cff`:

> Adu, B. *FamilyForge: pedigree-aware Mendelian and ACMG-style interpretation of a family callset.* Genomics & Bioinformatics Core, Noguchi Memorial Institute for Medical Research, University of Ghana, Legon, Ghana. Version 0.1.0, 2026.

**The forge family.** CaptureForge (design) → CallForge (analysis) → FamilyForge (interpretation). FamilyForge consumes CallForge's annotated joint VCF, `cnv_calls.tsv`, and `provenance.json`.

**Underlying resources** referenced by the interpretation logic and reports: Ensembl VEP (consequence, HGVS, MANE, SIFT, PolyPhen); gnomAD (population allele frequencies); ClinVar (clinical significance); PhyloP (conservation); and the finalist in-silico predictors CADD, REVEL, and AlphaMissense. The ACMG/AMP variant-interpretation guidelines (Richards et al., 2015) inform the pragmatic tiering rubric.